Tools for software development

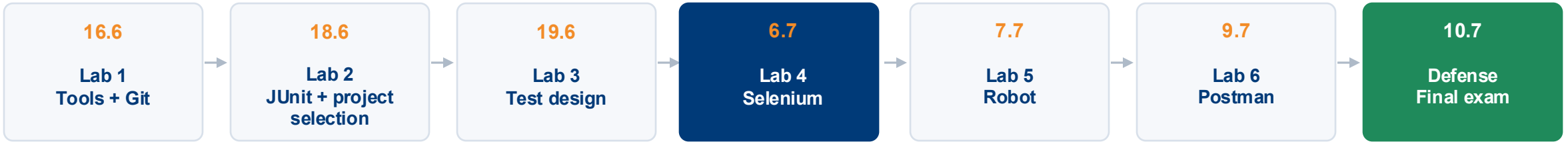
Lab 4 - **UI Test Automation with Selenium**

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Where we are in the course

The project moves from setup to professional test documentation.



Lab 4 outcomes

From manual test cases to automated UI checks

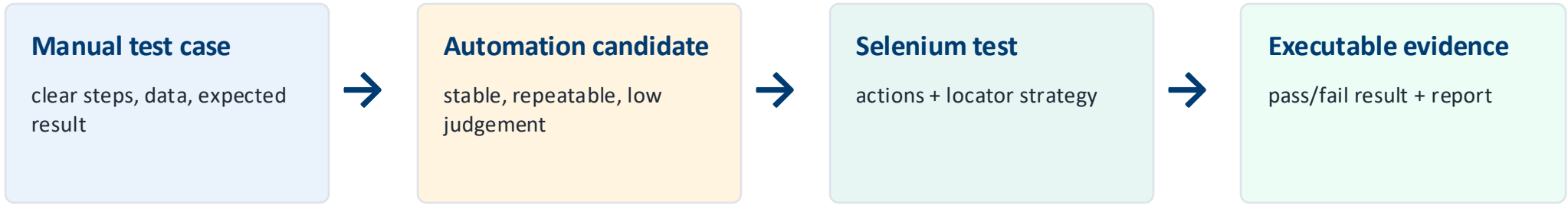
- Select manual test cases suitable for UI automation
- Record or implement Selenium tests for a real web application
- Add assertions to verify expected results
- Run a small test suite and save evidence
- Document what was automated, what failed, and what should stay manual

Expected team output

- 2 Selenium tests
- 1 assertion per test
- execution evidence
- automation report
- Git repository update

Lab 3 → Lab 4

Automation starts from already designed test cases



A Selenium test is only useful if it verifies something.

What is suitable for automation?

Choose repetitive and stable scenarios

Good candidates

- Login
- Search
- Add to cart
- Form validation
- Navigation
- Checkout without payment
- Regression tests

Risky candidates

- CAPTCHA
- 2FA
- Payment gateway
- Visual judgement
- Unstable external data
- Dynamic UI without stable selectors
- Manual decision required

Selenium toolkit

Pick the tool level that matches your team

Selenium IDE

- Browser extension
- Fast recording
- Useful for prototyping
- Low coding requirement

Katalon Recorder

- Selenium IDE not longer updated
- Recorder for selenium test scenariors

Selenium WebDriver

- Programmatic tests
- Java / Python / C# ...
- Better maintainability
- More control

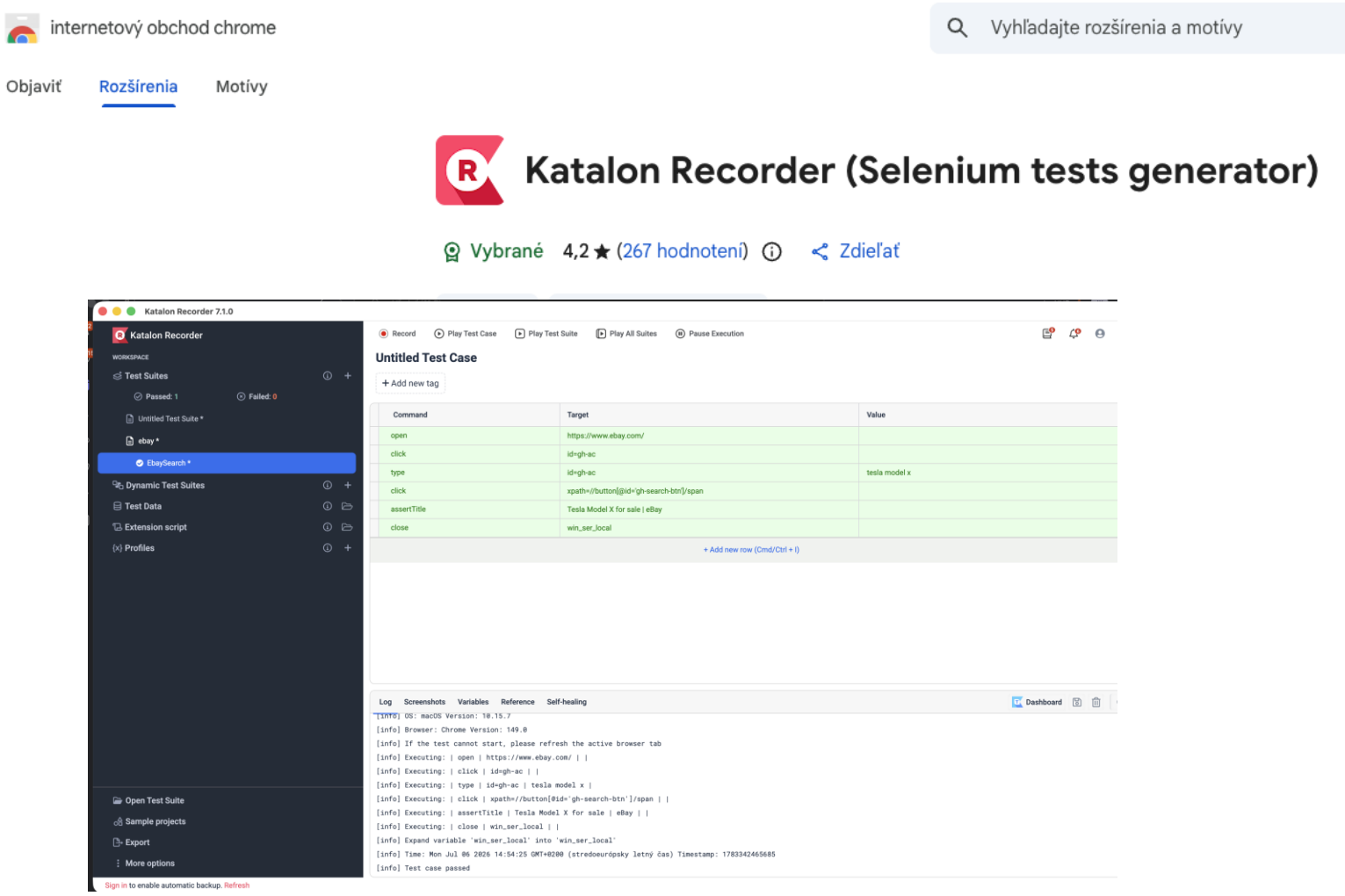
Selenium Grid

- Parallel execution
- Multiple browsers
- Multiple machines
- Advanced use case

For this lab: Katalon Recorder is enough. Stronger teams may use WebDriver.

Adding Katalon Recorder

Plugin to web browser



Katalon Recorder workflow

Record, improve, verify, run

1 → 2 → 3 → 4 → 5

Create project

set base URL

Start recording

perform scenario

Stop recording

clean commands

Add assertion

verify expected result

Run test suite

save evidence

Demo: valid login test

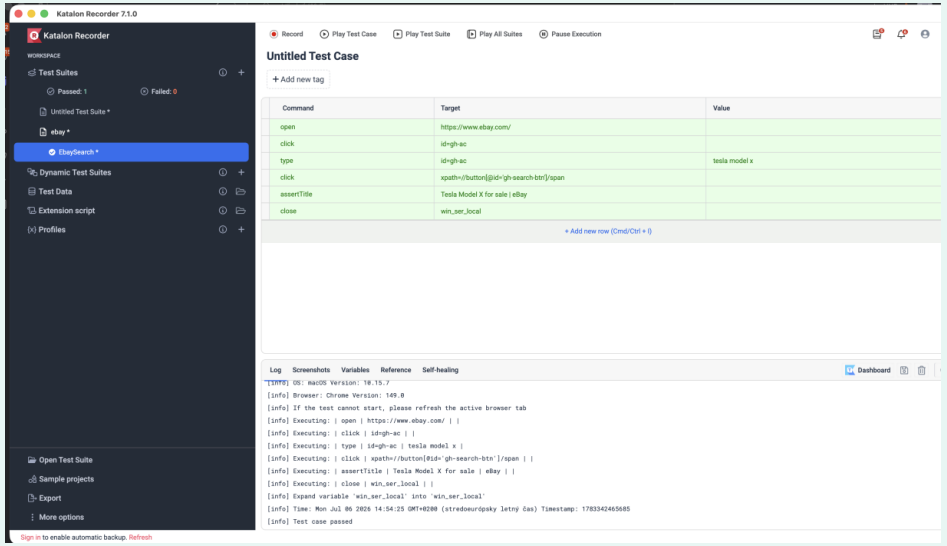
Example scenario using ebay.com

Scenario:

1. Open ebay.com
2. Search for “Tesla model x ”
3. On the searched page, check the **page name** to see if it matches the search query.
4. Verify that Product page is displayed

Expected result

User is logged in and the product list is visible.



Typical Selenium IDE commands

Actions must be followed by verification

Command	Purpose	Example
open	opens a page	/
type	fills input field	user-name = standard_user
click	clicks button or link	login-button
assert text	checks visible text	Products
assert element present	checks UI element	shopping cart badge

Rule: without assertion, it is not a real automated test.

Assertions: what should the test verify?

Every automated test needs a checkpoint

Scenario	Good assertion
Valid login	Products page or user dashboard is displayed
Invalid login	Error message is displayed
Add to cart	Cart badge or cart item is visible
Search	Result list contains expected keyword
Required field validation	Validation message appears

Exercise 1: select automation candidates

Start from your Lab 3 manual tests

- Open manual-tests/manual-test-cases.md
- Select at least 2 test cases suitable for Selenium automation
- Explain why each selected case is suitable
- Mark at least 1 case that should stay manual
- Save the decision table in automation/selenium/automation-selection.md

TC ID	Title	Automate?	Reason
TC-001	Valid login	Yes	stable and repetitive
TC-002	Invalid password	Yes	good regression test
TC-009	Visual layout check	No	requires human judgement

Exercise 2: create Selenium tests

Choose one variant based on your project

Variant A: Sauce Demo

Automate valid login and invalid login. Verify product page or error message.

Variant B: Demo Web Shop

Automate product search and verify the result page or product list.

Variant C: own app

Automate two manual test cases from your Lab 3 test suite.

Requirement: every test must contain at least one assertion.

Exercise 3: save, run and export evidence

Tests must be reusable, not only demonstrated once

Selenium IDE output

- Save project as:
- automation/selenium/team-name.side

- Save screenshots or execution evidence in reports/.

WebDriver output

- Save Java/Python test code under:
- automation/selenium/src/test/...

- Commit dependencies and instructions.

WebDriver example: valid login

For stronger teams demonstration ;)

```
def test_app_dynamics_job(self):
        driver = self.driver
        driver.get("https://www.ebay.com/")
        driver.find_element_by_id("gh-ac").click()
        driver.find_element_by_id("gh-ac").clear()
        driver.find_element_by_id("gh-
ac").send_keys("tesla model x")

driver.find_element_by_xpath("//button[@id='gh-
search-btn']/span").click()
        self.assertEqual("Tesla Model X for sale |
eBay", driver.title)
        driver.close()
```

Reading the WebDriver test

What does each part do?

Code part	Meaning
driver.get()	opens the tested web page
findElement()	finds a UI element by id, css selector, class, …
sendKeys()	enters data into input fields
click()	performs user action
assertTrue()	verifies expected result
driver.quit()	closes the browser after test execution

WebDriver example: invalid login

For stronger teams demonstration ;)

```
@Test
void shouldShowErrorForInvalidPassword() {
    WebDriver driver = new ChromeDriver();
    try {
        driver.get("https://www.saucedemo.com/");
        driver.findElement(By.id("user-name"))
              .sendKeys("standard_user");
        driver.findElement(By.id("password"))
              .sendKeys("wrong_password");
        driver.findElement(By.id("login-button")).click();
        WebElement error = driver.findElement(
              By.cssSelector("[data-test='error']"));
        assertTrue(error.getText().contains(
              "Username and password do not match"));
    } finally {
        driver.quit();
    }
}
```

Selenium WebDriver in IDE

- create classic Java code
  - run the test

HelloWorld.java

```java
package PokusnaApka;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public class HelloWorld {

    public static void main(String[] args) {

        System.setProperty("webdriver.firefox.driver","C:\\ Skola\\ZTS\\selenium\\geckodriver-v0.30.0-win64");
        WebDriver driver = new FirefoxDriver();
        driver.get("http://www.fri.uniza.sk");
        driver.quit();

    }
}
```

- create a Maven / Gradle project
  - run as a test

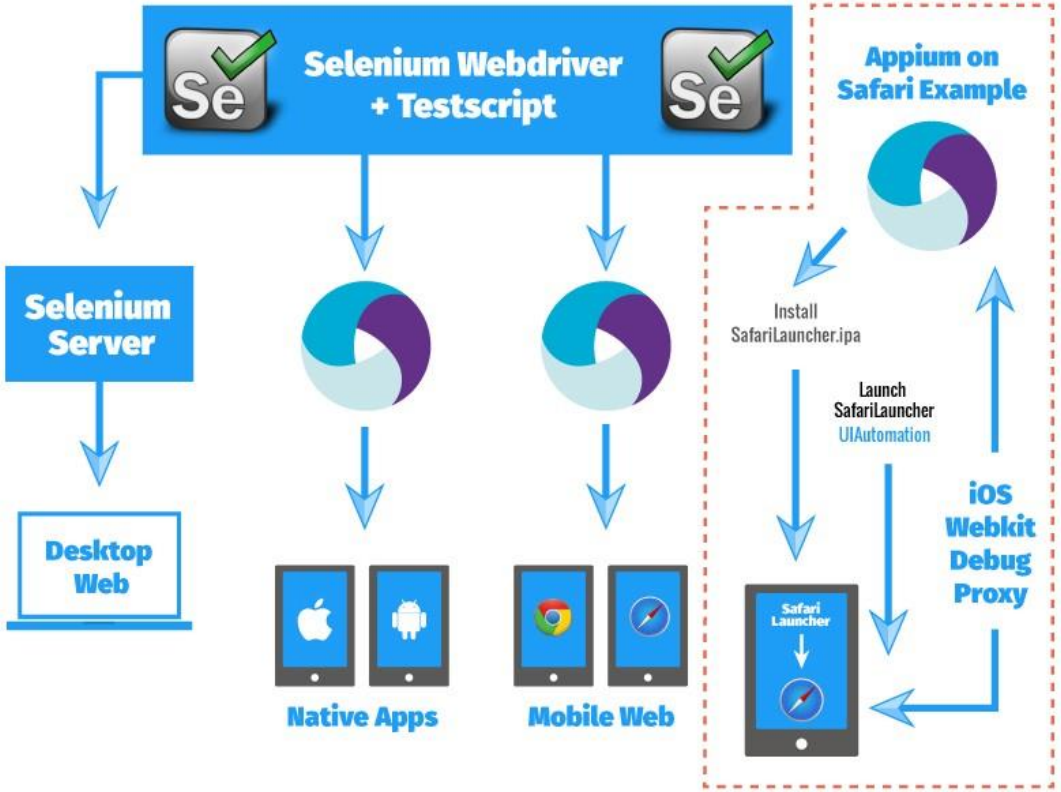
```xml
<dependencies>
    <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.0.0</version>
    </dependency>

</dependencies>
```

Selenium WebDriver - architecture

Selenium WebDriver – what I can't test

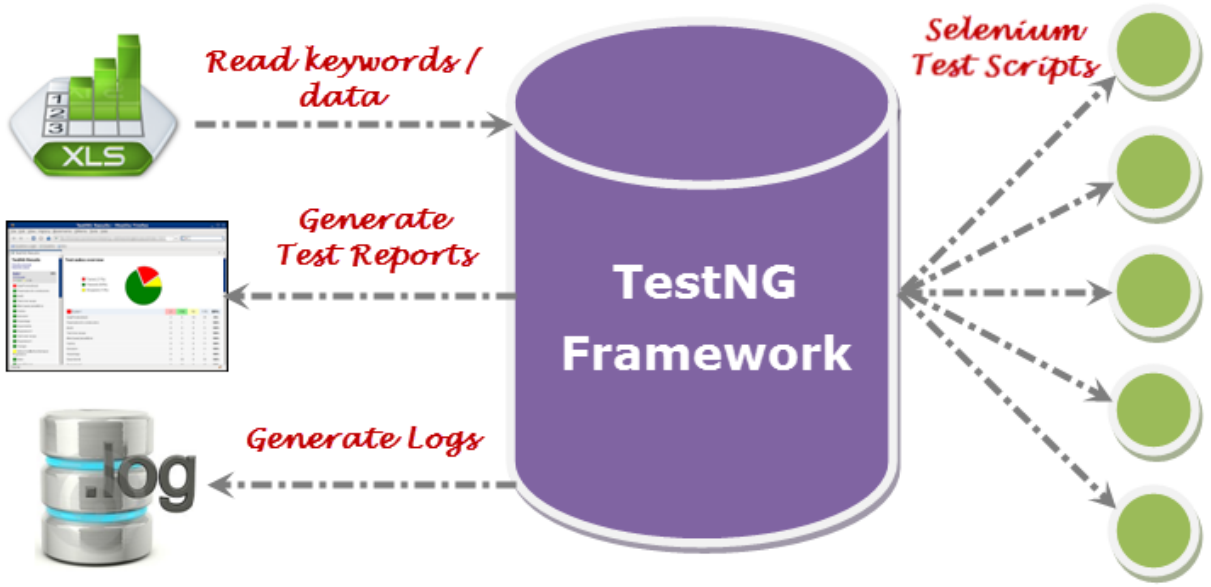
- it is not possible to test mobile applications
  - necessary need for a framework Appium



- possible testing of sequential tests
  - Grid for test parallelization

Selenium WebDriver – what I can't test

- limited report output
  - only with suppoert TestNG tool



- we cannot test images on websites

Locator strategy

Stable selectors make tests maintainable

Prefer

- id
- data-test
- name
- stable CSS selector
- explicit labels

Avoid when possible

- long absolute XPath
- visual position only
- random generated ids
- text that changes often
- classes used only for styling

Document problems

- If a locator is fragile, write it into the automation report. This is a useful testing finding.

Common automation problems

Real applications are asynchronous and dynamic

Problem	Typical cause	Good response
Element not found	page not loaded yet	wait or use stable selector
Test passes once, fails later	unstable data/UI	reduce dependencies
Wrong element clicked	locator too broad	make selector specific
Assertion fails	expected result unclear	fix test case or app

Repository structure after Lab 4

Keep test assets organized

```
automation/
└── selenium/
    ├── automation-selection.md
    ├── team-name.py
    └── README.md

reports/
└── lab4-selenium-report.md
```

Minimum repository content

- Selected test cases
- Selenium test file – export to python from Katalon Record
- Execution evidence
- Short report
- Run instructions

Commit messages should explain what changed.

Lab 4 automation report

Short, factual and useful

- Selected manual test cases and reason for automation
- Tool used: Katalon Recorder or Selenium WebDriver
- Execution results: passed / failed / blocked
- Evidence: screenshot, exported .py file, logs or test output
- Problems found: locator issues, timing, application defects
- What should remain manual and why

```
# Lab 4 - Selenium Automation Report

## Selected test cases
## Tool used
## Results
## Problems found
## What should remain manual
```

Lab 4 outcomes

From manual test cases to automated UI checks

- Select manual test cases suitable for UI automation
- Record or implement Selenium tests for a real web application
- Add assertions to verify expected results
- Run a small test suite and save evidence
- Document what was automated, what failed, and what should stay manual

Expected team output

- 2 Selenium tests
- 1 assertion per test
- execution evidence
- automation report
- Git repository update

Lab 4 checkpoint

Submit before the end of the lab

Team submits

- Repository URL
- Selected automated test cases
- Number of Selenium tests
- Tool used
- Assertions used
- Execution result
- Main problem during automation

Minimum output

- 2 Selenium tests
- 1 assertion per test
- 1 execution result screenshot/report
- 1 short automation report
- Updated Git repository

Suggested grading for Lab 4

10 points

Criterion	Points
Selection of suitable test cases	2
At least 2 Selenium tests	3
Assertions / checkpoints	2
Execution evidence	1
Repository documentation	1
Reflection: what not to automate and why	1
Total	10

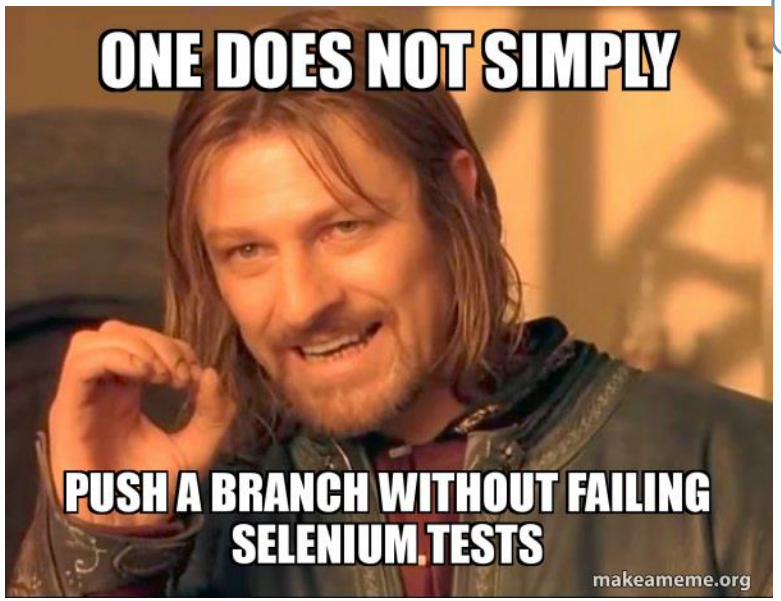
Preparation for Lab 5

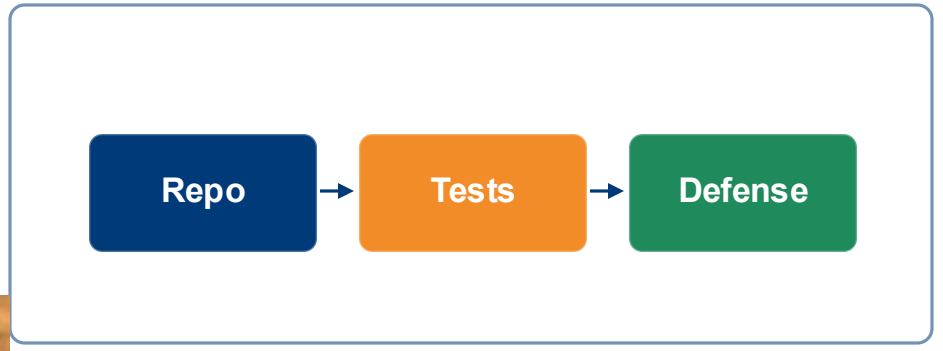
From Selenium scenario to keyword-based testing

- Select one Selenium scenario that can be rewritten using reusable actions
- Identify repeated steps: open page, enter data, click button, verify result
- Write these steps as keywords in plain English
- Prepare test data for the same scenario
- Bring Selenium test files and notes to the next lab

Example keywords

- Open Login Page
- Enter Username
- Enter Password
- Click Login
- Verify Login Success





Thank you for your attention.

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk